

Введение

Как мы уже знаем, список представляет собой **упорядоченную** последовательность элементов, пронумерованных от 00. Элементами списка могут быть любые типы данных – числа, строки, булевы значения и т.д. Например,

```
numbers = [10, 3]
```

```
constants = [3.1415, 2.71828, 1.1415]
```

```
countries = ['Russia', 'Armenia', 'Argentina']
```

```
flags = [True, False]
```

Список `numbers` состоит из 22 элементов, и каждый из них — **целое число**:

- `numbers[0] == 10;`
- `numbers[1] == 3.`

Список `constants` состоит из 33 элементов, каждый из которых — **вещественное число**:

- `constants[0] == 3.1415;`
- `constants[1] == 2.71828;`
- `constants[2] == 1.1415.`

Список `countries` состоит из 33 элементов, каждый из которых — **строка**:

- `countries[0] == 'Russia';`
- `countries[1] == 'Armenia';`
- `countries[2] == 'Argentina'.`

Список `flags` состоит из 22 элементов, и каждый из них — **булево значение**:

- `flags[0] == True;`
- `flags[1] == False.`

Мы также говорили, что элементы списка не обязательно должны иметь одинаковый тип данных. Список может содержать значения **разных типов данных**:

```
info = ['Timur', 1992, 72.5]
```

Список `info` содержит строковое значение, целое число и число с плавающей точкой:

- `info[0] == 'Timur';`

- `info[1] == 1992;`
- `info[2] == 72.5.`

Обычно элементы списка содержат данные одного типа и на практике редко приходится создавать списки содержащие элементы разных типов данных.

Вложенные списки

Оказывается, элементами списков могут быть другие списки и на практике такая конструкция оказывается очень полезной. Такие списки называются **вложенными списками**.

Создание вложенного списка

Работа с вложенными списками принципиально ничем не отличается от работы со списками, например, чисел или строк. Чтобы создать вложенный список, мы также перечисляем элементы через запятую в квадратных скобках:

```
my_list = [[0], [1, 2], [3, 4, 5]]
```

Переменная `my_list` ссылается на список, состоящий из других списков (с вложенными списками).

Поскольку глубина вложенности списка `my_list` равна двум, то такой список обычно называют **двумерным списком**. На практике, как правило, мы работаем с **двумерными списками**, реже с **трехмерными**.

Рассмотрим программный код:

```
my_list = [[0], [1, 2], [3, 4, 5]]

print(my_list)

print(my_list[0])

print(my_list[1])

print(my_list[2])

print(len(my_list))
```

Результатом работы такого кода будет:

```
[[0], [1, 2], [3, 4, 5]]
```

```
[0]
```

```
[1, 2]
```

```
[3, 4, 5]
```

```
3
```

Давайте взглянем на каждую строку приведенного кода поближе.

- строка 11 создает список и присваивает его переменной `my_list`. Список имеет три элемента, и каждый элемент тоже является списком:
- элементом `my_list[0]` является список `[0]`;
- элементом `my_list[1]` является список `[1, 2]`;
- элементом `my_list[2]` является список `[3, 4, 5]`.
- строка 22 распечатывает весь список `my_list`;
- строка 33 распечатывает элемент `my_list[0]`;
- строка 44 распечатывает элемент `my_list[1]`;
- строка 55 распечатывает элемент `my_list[2]`;
- строка 66 распечатает количество элементов списка `my_list`, то есть число 33.

Индексация

При работе с **одномерными списками** мы использовали **индексацию**, то есть обращение к конкретному элементу по его индексу. Аналогично можно индексировать и вложенные списки:

```
my_list = ['Python', [10, 20, 30], ['Beegreek', 'Stepik']]
```

```
print(my_list[0])
```

```
print(my_list[1])
```

```
print(my_list[2])
```

Результатом работы такого кода будет:

```
Python
```

```
[10, 20, 30]
```

```
['Beegreek', 'Stepik']
```

Так как элементы списка `my_list` – строка и списки, их также можно индексировать.

Рассмотрим программный код:

```
my_list = ['Python', [10, 20, 30], ['Beegreek', 'Stepik!']]
```

```
print(my_list[0][2])      # индексирование строки 'Python'
```

```
print(my_list[1][1])      # индексирование списка [10, 20, 30]
```

```
print(my_list[2][-1])      # индексирование списка ['Beegreek',  
'Stepik!']
```

```
print(my_list[2][-1][-1])  # индексирование строки 'Stepik!'
```

Результатом работы такого кода будет:

```
t
```

```
20
```

```
Stepik!
```

```
!
```

Попытка обратиться к элементу списка по несуществующему индексу:

```
print(my_list[3])          # у списка my_list индексы от 0 до 2
```

вызовет ошибку:

```
IndexError: index out of range
```

Функции len(), max(), min()

В прошлом курсе мы рассматривали встроенные функции `max()`, `min()`, `len()`, полезные и при работе с вложенными списками (обработке вложенных списков).

Функция len()

Рассмотрим программный код:

```
my_list = [[0], [1, 2], [3, 4, 5], [], [10, 20, 30]]
```

```
print(len(my_list))
```

Результатом работы такого кода будет:

```
5
```

Обратите внимание, встроенная функция `len()` возвращает количество элементов (число 55) списка `my_list`, а не общее количество элементов во всех списках (число 99).

Если требуется посчитать общее количество элементов во вложенном списке `my_list`, мы можем использовать цикл `for` в связке с функцией `len()`:

```
total = 0
```

```
my_list = [[0], [1, 2], [3, 4, 5], [], [10, 20, 30]]
```

```
for li in my_list:
```

```
    total += len(li)
```

```
print(total)
```

Результатом работы такого кода будет:

9

Переменная `li` последовательно принимает все значения элементов списка `my_list`, то есть является сама по себе списком, поэтому мы можем вызывать функцию `len()` с переданным аргументом `li`.

Функции `min()` и `max()`

Функции `min()` и `max()` могут работать и со списками. Если этим функциям передается несколько списков, то целиком возвращается один из переданных списков. При этом сравнение происходит поэлементно: сначала сравниваются первые элементы списков. Если они не равны, то функция `min()` вернет тот список, первый элемент которого меньше, `max()` – наоборот. Если первые элементы равны, то будут сравниваться вторые и т. д.

Рассмотрим программный код:

```
list1 = [1, 7, 12, 0, 9, 100]
```

```
list2 = [1, 7, 90]
```

```
list3 = [1, 10]
```

```
print(min(list1, list2, list3))
```

```
print(max(list1, list2, list3))
```

Результатом работы такого кода будет:

```
[1, 7, 12, 0, 9, 100]
```

```
[1, 10]
```

Функции `min()` и `max()` также можно использовать при работе с вложенными списками. Рассмотрим программный код:

```
list1 = [[1, 7, 12, 0, 9, 100], [1, 7, 90], [1, 10]]  
list2 = [['a', 'b'], ['a'], ['d', 'p', 'q']]
```

```
print(min(list1))  
print(max(list1))  
print(min(list2))  
print(max(list2))
```

Результатом работы такого кода будет:

```
[1, 7, 12, 0, 9, 100]  
[1, 10]  
['a']  
['d', 'p', 'q']
```

Обратите внимание – элементы вложенных списков в этой ситуации должны быть сравнимы.

Таким образом, следующий код:

```
my_list = [[1, 7, 12, 0, 9, 100], ['a', 'b']]  
  
print(min(my_list))  
print(max(my_list))
```

приведет к возникновению ошибки:

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

Примечания

Примечание 1. Независимо от вложенности списков, нам нужно помнить по возможности все **списочные методы**:

1. метод `append()` добавляет новый элемент в конец списка.
2. метод `extend()` расширяет один список другим списком.
3. метод `insert()` вставляет значение в список в заданной позиции.

4. метод `index()` возвращает индекс первого элемента, значение которого равняется переданному в метод значению.
5. метод `remove()` удаляет первый элемент, значение которого равняется переданному в метод значению.
6. метод `pop()` удаляет элемент по указанному индексу и возвращает его.
7. метод `count()` возвращает количество элементов в списке, значения которых равны переданному в метод значению.
8. метод `reverse()` инвертирует порядок следования значений в списке, то есть меняет его на противоположный.
9. метод `copy()` создает поверхностную копию списка.
10. метод `clear()` удаляет все элементы из списка.
11. оператор `del` позволяет удалять элементы списка по определенному индексу.

Примечание 2. Методы строк, работающие со списками:

1. метод `split()` разбивает строку на слова, используя в качестве разделителя последовательность пробельных символов, символ табуляции (`\t`) или символ новой строки (`\n`).
2. метод `join()` собирает строку из элементов списка, используя в качестве разделителя строку, к которой применяется метод.

Примечание 3. Язык Python не ограничивает нас в уровнях вложенности: элементами списка могут быть списки, их элементами могут быть другие списки, элементами которых в свою очередь могут быть другие списки...

Создание вложенных списков

Для создания вложенного списка можно использовать литеральную форму записи – перечисление элементов через запятую в квадратных скобках:

```
my_list = [[0], [1, 2], [3, 4, 5]]
```

Иногда нужно создать вложенный список, заполненный по определенному правилу – шаблону. Например, список длиной n , содержащий списки длиной m , каждый из которых заполнен нулями.

Рассмотрим несколько способов решения задачи.

Способ 1. Создадим пустой список, потом n раз добавим в него новый элемент – список длины m , составленный из нулей:

```
n, m = int(input()), int(input()) # считываем значения n и m
```

```
my_list = []
```

```
for _ in range(n):
```

```
    my_list.append([0] * m)
```

```
print(my_list)
```

Если ввести значения $n = 3, m = 5$, то результатом работы такого кода будет:

```
[[0, 0, 0, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 0, 0]]
```

Если передать значения $n = 5, m = 3$, то результатом работы такого кода будет:

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

Способ 2. Сначала создадим список из n элементов (для начала просто из n нулей). Затем сделаем каждый элемент списка ссылкой на другой список из m элементов, заполненный нулями:

```
n, m = int(input()), int(input()) # считываем значения n и m
```

```
my_list = [0] * n
```

```
for i in range(n):
```



```
my_list[i] = [0] * m
```

```
print(my_list)
```

Способ 3. Можно использовать генератор списка: создадим список из `n` элементов, каждый из которых будет списком, состоящих из `m` нулей:

```
n, m = int(input()), int(input()) # считываем значения n и m
```

```
my_list = [[0] * m for _ in range(n)]
```

```
print(my_list)
```

В этом случае каждый элемент создается независимо от остальных (заново конструируется вложенный список `[0] * m` для заполнения очередного элемента списка).

Обратите внимание, что очевидное решение, использующее операцию умножения списка на число (операция повторения) оказывается неверным:

```
n, m = int(input()), int(input()) # считываем значения n и m
```

```
my_list = [[0] * m] * n
```

```
print(my_list)
```

В этом легко убедиться, если присвоить элементу `my_list[0][0]` любое значение, например, `17`, а затем вывести список на печать:

```
n, m = int(input()), int(input())
```

```
my_list = [[0] * m] * n
```

```
my_list[0][0] = 17
```

```
print(my_list)
```

Если ввести значения `n = 5, m = 3`, то результатом работы такого кода будет:

```
[[17, 0, 0], [17, 0, 0], [17, 0, 0], [17, 0, 0], [17, 0, 0]]
```

То есть, изменив значение элемента списка `my_list[0][0]` мы также изменили значения

элементов `my_list[1][0], my_list[2][0], my_list[3][0], my_list[4][0]`.

Причина в самой природе списков (тип `list`). В Python списки – ссылочный тип данных. Конструкция `[0] * m` возвращает **ссылку** на список из `m` нулей. Повторение этого элемента создает список из `n` ссылок на один и тот же список.

Вложенный список нельзя создать при помощи операции повторения (умножения списка на число). Для корректного создания вложенного списка мы используем способы 1- 3, отдавая предпочтение способу 3.

Считывание вложенных списков

Если элементы списка вводятся через клавиатуру (каждая строка на отдельной строке, всего `n` строк, числа в строке разделяются пробелами), для ввода списка можно использовать следующий код:

```
n = 4 # количество строк
(элементов)

my_list = []

for _ in range(n):

    elem = [int(i) for i in input().split()] # создаем список из
элементов строки

    my_list.append(elem)
```

В этом примере мы используем списочный метод `append()`, передавая ему в качестве аргумента другой список. Так у нас получается список списков.

В результате, если на вход программе подаются строки

```
2 4
```

```
6 7 8 9
```

```
1 3
```

```
5 6 5 4 3 1
```

то в переменной `my_list` будет храниться список:

```
[[2, 4], [6, 7, 8, 9], [1, 3], [5, 6, 5, 4, 3, 1]]
```

Не забывайте, что метод `split()` возвращает **список строк, а не чисел**.

Поэтому мы предварительно сконвертировали строку в число, с помощью вызова функции `int()`.

Также, следует помнить отличие работы списочных методов `append()` и `extend()`.

Следующий код:

```
n = 4

my_list = []

for _ in range(n):

    elem = [int(i) for i in input().split()]

    my_list.extend(elem)
```

создает одномерный (!) список, а не вложенный. В переменной `my_list` будет храниться список:

```
[2, 4, 6, 7, 8, 9, 1, 3, 5, 6, 5, 4, 3, 1]
```

Перебор и вывод элементов вложенного списка

Как мы уже знаем для доступа к элементу списка указывают индекс этого элемента в квадратных скобках. В случае двумерных вложенных списков надо указать два индекса (каждый в отдельных квадратных скобках), в случае трехмерного списка — три индекса и т. д.

Рассмотрим программный код:

```
my_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

print(my_list[0][0])

print(my_list[1][2])

print(my_list[2][1])
```

Результатом работы такого кода будет:

```
1
6
8
```

Когда нужно перебрать все элементы вложенного списка (например, чтобы вывести их на экран), обычно используются **вложенные циклы**.

Рассмотрим программный код:

```
my_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

for i in range(len(my_list)):

    for j in range(len(my_list[i])):

        print(my_list[i][j], end=' ')    # используем необязательный
        параметр end

    print()                               # перенос на новую строку
```

Результатом работы такого кода будет:

```
1 2 3
```

```
4 5 6
```

```
7 8 9
```

Вызов функции `print()` с пустыми параметрами нужен для того, чтобы переносить вывод на новую строку, после того как будет распечатан очередной элемент (список) вложенного списка.

В предыдущем примере мы перебирали **индексы элементов**, а можно сразу перебирать сами элементы вложенного списка:

```
my_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

for row in my_list:

    for elem in row:

        print(elem, end=' ')

    print()
```

Результатом работы такого кода будет:

```
1 2 3
```

```
4 5 6
```

```
7 8 9
```

Перебор элементов вложенного списка по индексам дает нам больше гибкости для вывода данных. Например, поменяв порядок переменных `i` и `j` мы получаем иной тип вывода:

```
my_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

for i in range(len(my_list)):
    for j in range(len(my_list[i])):
        print(my_list[j][i], end=' ')    # выводим my_list[j][i]
        вместо my_list[i][j]
    print()
```

Результатом работы такого кода будет:

```
1 4 7
2 5 8
3 6 9
```

Обработка вложенных списков

Для обработки элементов вложенного списка, так же как и для вывода его элементов на экран как правило используются **вложенные циклы**.

Используем вложенный цикл для подсчета суммы всех чисел в списке:

```
my_list = [[1, 9, 8, 7, 4], [7, 3, 4], [2, 1]]

total = 0

for i in range(len(my_list)):
    for j in range(len(my_list[i])):
        total += my_list[i][j]

print(total)
```

Или то же самое с циклом не по индексу, а по значениям:

```
my_list = [[1, 9, 8, 7, 4], [7, 3, 4], [2, 1]]

total = 0
```

```
for row in my_list:

    for elem in row:

        total += elem

print(total)
```

Таким образом можно обработать элементы вложенного списка практически в любом языке программирования. В Python, однако можно упростить код, если использовать встроенную функцию `sum()`, которая принимает список чисел и возвращает его сумму. Подсчет суммы с помощью функции `sum()` выглядит так:

```
my_list = [[1, 9, 8, 7, 4], [7, 3, 4], [2, 1]]
```

```
total = 0

for row in my_list:      # В ОДИН ЦИКЛ

    total += sum(row)

print(total)
```

Названия переменных `row` (строка) и `elem` (элемент) удобно использовать при переборе вложенного списка по значениям. Названия переменных `i` и `j` используются при переборе вложенного списка по индексам.

Метод `ljust()`

Строковый метод `ljust()` выравнивает текст по ширине, **добавляя пробелы** в конец текста.

Результатом выполнения следующего кода:

```
print('a'.ljust(3))

print('ab'.ljust(3))

print('abc'.ljust(3))
```

будет:

```
a__
ab_
abc
```

Исходная строка не обрезается, даже если в ней больше символов, чем нужно.

Результатом выполнения следующего кода:

```
print('abcdefg'.ljust(3))
```

будет:

```
abcdefg
```

Строковый метод `ljust()` использует вместо пробела другой символ, если передать ему второй аргумент, необязательный.

Результатом выполнения следующего кода:

```
print('a'.ljust(5, '*'))
```

```
print('ab'.ljust(5, '$'))
```

```
print('abc'.ljust(5, '#'))
```

будет:

```
a****
```

```
ab$$$
```

```
abc##
```

Метод `rjust()`

Строковый метод `rjust()` выравнивает текст по ширине, **добавляя пробелы** в начало текста.

Результатом выполнения следующего кода:

```
print('a'.rjust(3))
```

```
print('ab'.rjust(3))
```

```
print('abc'.rjust(3))
```

будет:

```
__a
```

```
__ab
```

```
abc
```

Исходная строка не обрезается, даже если в ней больше символов, чем нужно.

Результатом выполнения следующего кода:

```
print('abcdefg'.rjust(3))
```

будет:

```
abcdefg
```

Строковый метод `rjust()` использует вместо пробела другой символ, если передать ему второй аргумент, необязательный.

Результатом выполнения следующего кода:

```
print('a'.rjust(5, '*'))  
print('ab'.rjust(5, '$'))  
print('abc'.rjust(5, '#'))
```

будет:

```
****a  
$$$$ab  
##abc
```

Применив метод `ljust()` для выравнивания столбцов при выводе таблицы мы получим следующий код:

```
rows, cols = 3, 4          # rows - количество строк, cols -  
количество столбцов
```

```
matrix = [[277, -930, 11, 0],  
           [9, 43, 6, 87],  
           [4456, 8, 290, 7]]
```

```
for r in range(rows):  
    for c in range(cols):  
        print(str(matrix[r][c]).ljust(6), end=' ')  
    print()
```

Результатом выполнения такого кода будет:

```
277   -930   11     0  
9      43     6    87  
4456   8     290    7
```


